
THINKING DEEPER, NOT LONGER: MEMORY-EFFICIENT TEST-TIME REASONING WITH DEPTH-RECURRENT TRANSFORMERS FOR COMPOSITIONAL GENERALIZATION

Hung-Hsuan Chen

Computer Science and Information Engineering
National Central University
Taoyuan, Taiwan
hhchen1105@acm.org

ABSTRACT

Standard Transformers have a fixed computational depth, limiting their ability to generalize to tasks that require variable-depth reasoning. The usual remedy, Chain-of-Thought (CoT), spends tokens to reason, inflating the key-value cache and making latency grow with the step count, so memory becomes the limiting cost when reasoning is served over large query batches. We study a depth-recurrent Transformer that decouples computational depth from parameter count by iterating a shared-weight block, so that each added reasoning step costs flat memory and linear latency, with no token generation. Three ingredients keep the recurrence stable for 20+ thinking steps: a silent thinking objective that supervises only the final output, LayerScale initialization, and an identity-biased gate that opens a gradient highway across steps. We characterize it on three compositional domains with decreasing structural bias: graph reachability (adjacency masking), nested boolean logic (relative positioning), and unstructured relational text (no positional cue). We find a *computational frontier*: accuracy climbs once the thinking-step count meets the task’s complexity, reaching near-perfect performance on the two structured tasks and a lower plateau on unstructured text. How it climbs depends on the structural bias—abruptly from chance on the graph task, gradually on the other two. Depth recurrence extrapolates beyond the training range: it succeeds on the graph task where fixed-depth models barely extrapolate, and on the two sequence tasks comes within two points of fixed-depth Transformers that use 4–6.4 $\times$ more parameters. On the graph task, whose adjacency mask makes propagation depth verifiable, intermediate per-step supervision—a standard recipe for deep iterative models—consistently *harms* this extrapolation. We release the code for reproducibility.¹

Keywords Transformers · Recurrent Neural Networks · Compositional Generalization · Reasoning · Test-Time Computation

1 Introduction

Large Language Models (LLMs) have achieved remarkable performance across tasks. However, their reasoning capabilities remain constrained by architecture. When faced with problems that require multi-step logical deduction, such as planning or mathematical proof, current models rely heavily on *Chain-of-Thought* (CoT) prompting (Wei et al., 2022), which externalizes reasoning as a sequence of generated tokens. We call this paradigm *horizontal* CoT: the model “thinks” by extending the output sequence horizontally, consuming the available context length with each reasoning step.

Although horizontal CoT is effective, it suffers from fundamental limitations. First, each reasoning step costs tokens, consuming the finite context window. Second, the depth of computation available at each token position is fixed by the number of Transformer layers, regardless of problem difficulty. Third, generating intermediate reasoning as discrete

¹<https://github.com/hhchen1105/Depth-Recurrent-Transformer>

tokens introduces a distinct failure mode: each sampled token is a discrete symbol drawn from a finite vocabulary, collapsing the model’s continuous internal representation into a single categorical choice that all subsequent tokens must condition on. A continuous latent vector carries no such limitation: each entry remains real-valued, so later computation may reweight or attenuate an early error. Moreover, because horizontal CoT is typically trained with per-token supervision, an early token that appears promising may not be optimal for the final answer.

In this work, we study an orthogonal paradigm that we call *vertical* Chain-of-Thought: instead of generating tokens horizontally, the model “thinks deeper” by recurrently applying a shared-weight Transformer block. This approach decouples computational depth from both parameter count and context length. The model can invest more computation in harder instances by increasing the recurrence steps during inference, without generating additional tokens or consuming additional context window space. Because no tokens are emitted, the key–value cache does not grow with reasoning depth. This matters most when reasoning is served over large query batches, since every query carries its own cache. On a depth-24 backbone, the cache for 64 thinking tokens alone measures two to three times our entire recurrent core, and it keeps growing with each further step while our core’s footprint stays fixed (Section 5.4).

A practical concern in making depth recurrence work is stability. Unrolling a single shared-weight block for many steps is classically prone to exploding or vanishing gradients (Pascanu et al., 2013) and representation collapse (Dong et al., 2021). However, a plain weight-tied block reaches competitive accuracy in our tasks (Section 4.4); what it lacks is stability: in one of our tasks, out-of-distribution (OOD) accuracy ranges from 62 to 76 across random seeds. We obtain more reliable optimization and better-behaved gradients from three complementary mechanisms (Section 3):

1. **Silent Thinking:** We apply supervision *only* at the final recurrence step, with no intermediate auxiliary losses. This forces the model to develop genuine multi-step reasoning paths rather than learning heuristic shortcuts.
2. **LayerScale Initialization:** Per-channel scaling initialized at 10^{-4} after attention and feed-forward layers, preventing initial random weights from corrupting preserved hidden states during early training.
3. **Identity-Biased Recurrence:** To improve gradient propagation when unrolling for 20+ recurrence steps, we use a gated recurrence with the gate bias initialized to -2.0 , strongly biased toward preserving the previous hidden state. This creates a gradient highway across recurrence steps (analyzed in Section 5.3).

To evaluate the Depth-Recurrent Transformer’s capabilities, we design three OOD reasoning tasks. We begin with *Graph Reachability*, which provides a physically verifiable testbed using topological masking. We then increase the structural complexity with *Nested Boolean Logic*, demonstrating the model’s ability to maintain hierarchical states using relative positioning. Finally, we remove all task-aligned structural inductive biases in a *Relational Composition* task over unstructured text, showing that our invariant reasoning core can discover complex latent routing paths in natural language. We choose these tasks because their computational depth is controllable, enabling rigorous analysis of generalization behavior.

Our experiments reveal a consistent pattern that we term *the computational frontier*: a diagonal boundary in the accuracy heatmap (thinking steps $\times$ task complexity) where performance transitions from chance-level to near-perfect on the two structured tasks and to a lower plateau on unstructured text. We attribute these different generalization profiles to the different inductive biases of their respective perception interfaces.

Our contributions are summarized as follows:

- We frame latent depth recurrence as a way to add test-time reasoning compute at a memory cost that is flat, rather than linear, in the number of reasoning steps, and give the instantiation: a shared-weight reasoning core of under 1M parameters, trained via silent thinking, LayerScale initialization, and identity-biased gating.
- Our test-time depth recurrence extrapolates to problem depths and step counts beyond the training setup. On the graph task, it generalizes where fixed-depth Transformers collapse, and on the less-structured tasks, it comes within two points of fixed-depth models that use $4\text{--}6.4\times$ the parameters.
- We identify and analyze the computational frontier phenomenon and show how task-specific perception interfaces yield qualitatively different generalization behaviors.
- We show that intermediate per-step supervision, a standard recipe for training deep iterative models, *harms* OOD generalization on the graph task, whose adjacency mask lets us verify how far information has actually propagated.
- We analytically and empirically show that the recurrent core holds peak memory flat and latency linear in the reasoning-step count, in contrast to the growing key–value cache and $\Theta(TN)$ critical path of horizontal CoT.

2 Related Work

2.1 Chain-of-Thought and Test-Time Compute

Chain-of-Thought prompting (Wei et al., 2022) and its variants (Kojima et al., 2022; Wang et al., 2023) have become the dominant paradigm for LLM reasoning. Recent work on test-time computing (Snell et al., 2025) demonstrates that allowing models to perform more inference-time computation improves performance. However, all these approaches operate via horizontal token generation, consuming a context window proportional to the reasoning depth. Our work explores an orthogonal axis, vertical depth recurrence in latent space, which achieves test-time compute scaling without token overhead.

2.2 Pause Tokens and Latent Reasoning

Researchers proposed appending learnable “pause tokens” to the input (Goyal et al., 2024), giving the Transformer additional forward-pass computation before producing output; Coconut (Hao et al., 2025) takes this further by feeding the last hidden state back as the next input embedding, keeping intermediate “thought” steps continuous rather than discretized into vocabulary tokens. Both share our motivation of extra inference-time computation, and Coconut further shares our goal of computing in continuous space. However, both remain fundamentally *horizontal*. Our approach instead increases depth by repeatedly applying the same block without growing the sequence. Unlike traditional RNNs that compress the sequence into a single bottleneck vector $h \in \mathbb{R}^d$, we maintain a full-sequence-length state matrix $H \in \mathbb{R}^{L \times d}$ at every step, preserving rich spatial interactions.

2.3 Universal Transformers

The Universal Transformer (UT) (Dehghani et al., 2019) introduced weight sharing across layers and used Adaptive Computation Time (ACT) (Graves, 2016) for dynamic stopping. Our work builds on this foundation but differs in three critical aspects. First, LayerScale (Touvron et al., 2021) protects latent representations during early training. Second, an identity-biased gated recurrence with negative bias initialization improves gradient propagation across many recurrence steps (Section 5.3). Third, rather than ACT’s complex token-level halting probabilities, we treat the recurrence step count T as an externally specified budget that decouples computational depth from optimization, supporting flexible test-time compute scaling without optimization overhead. This choice is not merely a simplification: we found ACT Universal Transformer learns a halting budget that often saturates below the depth OOD instances require, collapsing to chance on graph reachability beyond the training range (Sections 4.4 and 5.5).

2.4 Weight Sharing and Fixed-Point Views of Depth

Parameter sharing across layers has also been explored outside the recurrent-reasoning setting. ALBERT (Lan et al., 2020) shares parameters across layers as a model-compression technique. However, it applies the same fixed number of layers to every input, with no mechanism to vary that count at test time. A complementary perspective comes from Deep Equilibrium Models (DEQ) (Bai et al., 2019), which reinterpret weight-sharing recurrence as a fixed point solved via root-finding rather than unrolling. We retain the intermediate iterates, which lets us control how the number of iterations T trades off against accuracy.

2.5 Depth and Expressiveness in Transformers

Theoretical works have established that Transformer depth is critical in expressiveness: fixed-depth Transformers are limited to TC^0 circuit complexity (Merrill and Sabharwal, 2024), which excludes inherently sequential computations. Researchers demonstrated that depth-efficient Transformers cannot solve certain compositional tasks without sufficient layers (Feng et al., 2023). By making depth as a dynamic variable through recurrence, our architecture bypasses the TC^0 limitation: the unrolled reasoning steps can scale with the input’s intrinsic sequential complexity, enabling the model to solve inherently sequential tasks, such as multi-hop routing and nested logic, that fixed-depth networks struggles.

2.6 Summary Comparison

Table 1 compares our approach with the closest families of test-time-compute methods along the axes discussed above.

Table 1: Comparison of approaches for scaling test-time computation.

Method	Compute Axis	Weight Sharing	Supervision	Context Cost	Stability Mechanism
Horizontal CoT (Wei et al., 2022)	Token generation	No	Per-token / outcome	Grows with reasoning length	N/A (fixed depth per token)
Pause Tokens (Goyal et al., 2024) / Coconut (Hao et al., 2025)	Token generation (latent)	No	Per-token / outcome	Grows with # pause tokens	N/A (fixed depth per token)
Universal Transformer (Dehghani et al., 2019)	Layer recurrence	Yes	Final output + ponder cost (Graves, 2016)	None	ACT halting regularization
Ours	Layer recurrence	Yes	Final output	None	LayerScale + identity-biased gate

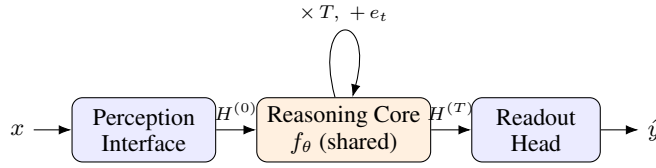


Figure 1: Overview of the depth-recurrent architecture. A task-specific perception interface encodes the input into $H^{(0)}$; the task-invariant core f_θ refines this state recurrently for T steps using a *single* set of shared weights, conditioned at each step on a depth embedding e_t ; a task-specific readout head decodes $H^{(T)}$ into the prediction $\hat{y}$. Unlike horizontal CoT, T can be scaled at inference time without adding parameters or lengthening the input/output sequence.

3 Method

Our architecture consists of three components (Figure 1): a *task-invariant reasoning core* that iteratively refines a hidden representation through shared-weight recurrence, a *task-specific perception interface* that encodes the raw input into this representation’s initial state, and a *task-specific readout head* that extracts the final prediction after T recurrence steps (T can be flexibly scaled at inference time).

3.1 The Invariant Reasoning Core

The reasoning core is a single Transformer block f_θ with shared weights, applied recurrently for T steps. Let L be the sequence length and d be the hidden dimension; the core operates on a hidden state $H^{(t)} \in \mathbb{R}^{L \times d}$, where the initial state $H^{(0)}$ is the output of the task-specific perception interface (Section 3.2). At each step $t \in \{1, \dots, T\}$, the candidate hidden state $\tilde{H}^{(t)} \in \mathbb{R}^{L \times d}$ is computed as:

$$\tilde{H}^{(t)} = f_\theta(H^{(t-1)}, t), \quad (1)$$

To ensure that this recurrent core can stably unroll for 20+ steps and learn genuine algorithms, we detail its architecture below and design it around three additional stability mechanisms.

Architecture of f_θ The block is a single Pre-LayerNorm Transformer encoder block (Xiong et al., 2020). At recurrence step t , it receives the depth-conditioned input $\hat{H}^{(t)} = H^{(t-1)} + e_t$, where e_t is the depth embedding described below. The block applies two sub-layers in sequence.

Sub-layer 1: Multi-Head Self-Attention (MHSA).

$$H' = \hat{H}^{(t)} + \gamma_{\text{attn}} \odot \text{MHSA}\left(\text{LN}\left(\hat{H}^{(t)}\right), M\right), \quad (2)$$

where $\text{LN}(\cdot)$ is Layer Normalization, M is the task-specific attention mask (the adjacency mask for the topological interface, or no mask for the other two; see Section 3.2), and $\gamma_{\text{attn}} \in \mathbb{R}^d$ is a LayerScale vector. The attention

Algorithm 1 Training Step with Silent Thinking

Require: input x , label y , perception interface Perc , readout head Read , reasoning block f_θ , training step range $[T_{\min}, T_{\max}^{\text{tr}}]$

- 1: $H^{(0)} \leftarrow \text{Perc}(x)$
- 2: **Sample** $T \sim \mathcal{U}\{T_{\min}, \dots, T_{\max}^{\text{tr}}\}$
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $\hat{H}^{(t)} \leftarrow H^{(t-1)} + e_t$ ▷ depth embedding
- 5: $\tilde{H}^{(t)} \leftarrow f_\theta(\hat{H}^{(t)})$
- 6: $z^{(t)} \leftarrow \sigma\left([\tilde{H}^{(t)}; H^{(t-1)}]W_z + b_z\right)$
- 7: $H^{(t)} \leftarrow z^{(t)} \odot \tilde{H}^{(t)} + (1 - z^{(t)}) \odot H^{(t-1)}$ ▷ no loss is computed here
- 8: **end for**
- 9: $\hat{y} \leftarrow \text{Read}(H^{(T)})$
- 10: $\mathcal{L} \leftarrow \text{CE}(\hat{y}, y)$ ▷ loss only at the final step T
- 11: Backpropagate $\mathcal{L}$ through the unrolled computation graph and update θ

mechanism computes h parallel heads. For each head i :

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}} + M\right) V_i, \quad (3)$$

$$Q_i = \hat{H}^{(t)} W_i^Q, \quad K_i = \hat{H}^{(t)} W_i^K, \quad V_i = \hat{H}^{(t)} W_i^V, \quad (4)$$

with $d_k = d/h$. For the hierarchical and unstructured interfaces, Rotary Position Embeddings (RoPE) (Su et al., 2024) are applied to the query and key projections before the dot product, so that relative position is encoded (see Section 3.2).

Sub-layer 2: Feed-Forward Network (FFN).

$$H'' = H' + \gamma_{\text{ffn}} \odot \text{FFN}(\text{LN}(H')), \quad (5)$$

$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2, \quad (6)$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$, and d_{ff} is the feedforward dimension. The output of f_θ is $\tilde{H}^{(t)} = H''$, which is subsequently combined with $H^{(t-1)}$ through the identity-biased gate (below) to produce $H^{(t)}$.

For the hierarchical and unstructured perception interfaces (Section 3.2), we apply LayerScale (Touvron et al., 2021) after each sub-layer, with $\gamma_{\text{attn}}, \gamma_{\text{ffn}} \in \mathbb{R}^d$ initialized to 10^{-4} and learned jointly with all other parameters. For the topological interface, where the hard adjacency mask strongly constrains the computation, we omit LayerScale and instead use the standard PyTorch TransformerEncoderLayer to replace the custom RoPE-integrated attention described above. Table 2 in Section 4 summarizes the architectural hyperparameters.

Silent Thinking Objective A fundamental design choice is how to supervise the recurrent steps. Instead of forcing intermediate answers or applying per-step auxiliary losses, we compute the cross-entropy (CE) loss $\text{CE}(\hat{y}, y)$ *only* on the final thinking step T .

During training, T is sampled uniformly from a task-specific range (e.g., $T \sim \mathcal{U}(1, 12)$). This randomized depth acts as a regularizer, encouraging the network to produce stable outputs across a range of computation depths, while the silent thinking objective forces the model to develop its own genuine multi-step reasoning paths. Algorithm 1 summarizes one training step. At inference time, T is set to any desired budget up to $T_{\max}$, independent of the range used during training.

Pre-Layer Normalization and LayerScale For complex symbolic tasks requiring a high step count, the accumulated variance from repeated sub-layer transformations can destabilize early training. Beyond the Pre-LayerNorm (Xiong et al., 2020) placement above, initializing the LayerScale vectors $\gamma_{\text{attn}}, \gamma_{\text{ffn}}$ near zero (10^{-4}) forces the early-training dynamics of f_θ to act almost perfectly as an identity mapping, protecting fragile boolean states from the destructive noise introduced by randomly initialized deep layers. As training progresses, the network selectively scales up γ to activate its reasoning capacity.

Identity-Biased Gated Recurrence To unroll a network for 20+ steps without vanishing gradients, we combine the new candidate representation $\tilde{H}^{(t)}$ with the previous state $H^{(t-1)}$ through a learned GRU-like gate:

$$z^{(t)} = \sigma\left([\tilde{H}^{(t)}; H^{(t-1)}] \cdot W_z + b_z\right) \quad (7)$$

$$H^{(t)} = z^{(t)} \odot \tilde{H}^{(t)} + (1 - z^{(t)}) \odot H^{(t-1)} \quad (8)$$

Here, $[\tilde{H}^{(t)}; H^{(t-1)}] \in \mathbb{R}^{L \times 2d}$ denotes the feature-dimension concatenation of the two states, and $W_z \in \mathbb{R}^{2d \times d}$ projects it to yield $z^{(t)} \in \mathbb{R}^{L \times d}$. The gate operates element-wise at each sequence position.

We initialize the gate bias $b_z = -2.0$. Because $\sigma(-2.0) \approx 0.12$, the model defaults to retaining 88% of its previous state. This physically guarantees that the initial signal decays slowly, enabling stable signal propagation through deep unrolling. We formalize this intuition with a gradient analysis in Section 5.3.

Depth Embeddings To prevent the shared-weight block f_θ from losing its place within the recurrent loop, we inject a learned depth embedding $e_t \in \mathbb{R}^d$ at each step $t \in \{1, \dots, T\}$ before the attention computation:

$$\hat{H}^{(t)} = H^{(t-1)} + \mathbf{1}_L \cdot e_t^\top, \quad (9)$$

where $\mathbf{1}_L \in \mathbb{R}^L$ is an all-one vector, so that e_t is broadcast identically across all L sequence positions. The embeddings $\{e_t\}_{t=1}^{T_{\max}}$ form a standard learned lookup table $E \in \mathbb{R}^{T_{\max} \times d}$, where $T_{\max}$ is set to cover the maximum number of steps used at both training and test time.

Without depth embeddings, the shared block f_θ has no signal indicating which recurrence step it is on, making it difficult to distinguish early exploratory steps from later refinement steps. Depth embeddings are orthogonal to the positional encodings used within the sequence (RoPE or none): the former encodes *which recurrence step* the state is currently at, the latter encodes *which sequence position* a token occupies, and both are applied simultaneously without conflict.

3.2 Task-Specific Perception Interfaces

Rather than hardcoding task-specific rules into the reasoning core, we design modular perception interfaces that inject varying degrees of structural inductive bias into the initial state $H^{(0)}$. We explore three distinct interface paradigms:

Topological Masking (Strict Structural Priors) To enforce physical constraints where computation must strictly follow defined topological pathways (e.g., graph routing), we introduce an adjacency mask. Given an adjacency matrix $A \in \{0, 1\}^{L \times L}$ over the L sequence positions, we add self-loops ($A_{ii} = 1$) and construct an additive mask matrix $M \in \mathbb{R}^{L \times L}$:

$$M_{ij} = \begin{cases} 0, & \text{if } A_{ij} = 1 \\ -\infty, & \text{if } A_{ij} = 0 \end{cases} \quad (10)$$

The attention is computed as $\text{softmax}(QK^T / \sqrt{d_k} + M)V$. The $-\infty$ term forces the attention probability to zero for non-adjacent nodes.

Relative Positional Encoding (Hierarchical Priors) For sequences where structural hierarchy depends on relative distances rather than strict adjacency (e.g., evaluating nested symbolic expressions), topological masking is overly restrictive. Instead, we utilize *Rotary Position Embeddings (RoPE)* (Su et al., 2024). By multiplying the query and key vectors with rotation matrices corresponding to their respective absolute positions m and n ($R_{\Theta, m}$ and $R_{\Theta, n}$), the inner product inherently encodes their relative sequence distance ($m - n$):

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = q^T R_{\Theta, m-n} k \quad (11)$$

Standard Sequence Attention (Unstructured / Task-Agnostic Priors) To test the reasoning core’s ability to autonomously discover latent routing paths in unstructured sequences (e.g., natural language facts), we remove task-specific structural biases. Although we retain standard Rotary Position Embeddings (RoPE) to let the perception interface process local word order, we completely shuffle the input facts. Consequently, unlike the previous experiment, where relative distance correlates with hierarchical depth, the 1D sequence distance in this bag-of-facts provides no meaningful structural hints about the underlying relational graph. The invariant reasoning core must discover the correct pointer-chasing routes entirely on its own.

Table 2: Architectural hyperparameters for each experiment. Depth-embedding capacity is the size of the learned lookup table E (an architectural upper bound, not itself an evaluated step count); the maximum step count actually evaluated in Sections 4.1–4.3 is 20 (graph), 24 (nested expr.), and 20 (family).

	Graph	Nested Expr.	Family
d	128	256	256
h (heads)	4	8	8
d_{ff}	256	1024	1024
RoPE	×	✓	✓
LayerScale	×	✓	✓
Gate bias b_z	−2.0	−2.0	−2.0
Train step range T	[5, 8]	[4, 16]	[1, 12]
Depth-emb. capacity	20	28	20

3.3 Task-Specific Readout Mechanisms

After unrolling the latent reasoning for T steps, a readout head decodes the final state $H^{(T)}$. We employ distinct readout mechanisms depending on the interface:

- Pairwise Node Readout (for the topological interface): The d -dimensional representations corresponding to the specific source and target nodes are extracted, concatenated, and passed through an MLP.
- Global Sequence Readout (for the hierarchical and unstructured interfaces): The global representation of the [CLS] token at position 0 is extracted and passed through an MLP. Because neither interface exposes the query positions to the readout, the reasoning core must route the relevant information to the [CLS] token by the final step.

4 Experiments

We evaluate our architecture on three compositional reasoning domains with decreasing structural inductive biases: graph reachability, Boolean logic, and relational text. The maximum number of evaluated reasoning steps varies across these tasks, reflecting the natural upper bounds of their respective structural complexities.

Table 2 summarizes the architectural hyperparameters used for each experiment. Graph reachability’s small node count requires less representational capacity than the richer symbolic and lexical vocabularies of the logic and family tasks, so we use a narrower d for efficiency.

4.1 Experiment I: Graph Reachability

Given a directed graph $G = (V, E)$, this task determines whether there exists a directed path from node s to r . The model is trained on instances requiring 1–5 hops with 5–8 thinking steps and is evaluated up to 12 hops to test OOD generalizability. Throughout this section, each heatmap marks both the depth and thinking-step training boundaries with dashed lines, with axes inside/outside those ranges denoting ID/OOD evaluation.

As illustrated in Figure 2, we observe a sharp computational frontier. The accuracy transition from chance to perfect/near-perfect: once given enough ($N+$) thinking steps, the model solves N -hop queries. One step fewer and accuracy drops to random. The model achieves 100% OOD generalization up to 8 hops ($1.6\times$), but collapses abruptly at 10 hops, indicating a generalization boundary enforced by the topological masking. This boundary is capacity-dependent: a wider model propagates several hops further (Section 4.5). In the thinking-step dimension, the model also generalizes stably both below (1–3 thinking steps) and above (12–20 thinking steps) the training range, with the diagonal frontier shifting accordingly.

Why Is the Frontier Sharp? Consider the 5-node chain graph $s \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow r$ (a 4-hop query). Under the adjacency mask M (Section 3.2), each application of f_θ moves information across at most one edge, so r ’s hidden state reflects s ’s existence only once $t \geq 4$; since the pairwise readout (Section 3.3) queries s and r directly, it can only report “reachable” once this frontier physically arrives, which is exactly why accuracy collapses to chance one step below the required hop count. Once it arrives, the identity-biased gate (Section 3.1) favors retaining it, so extra steps rarely hurt.

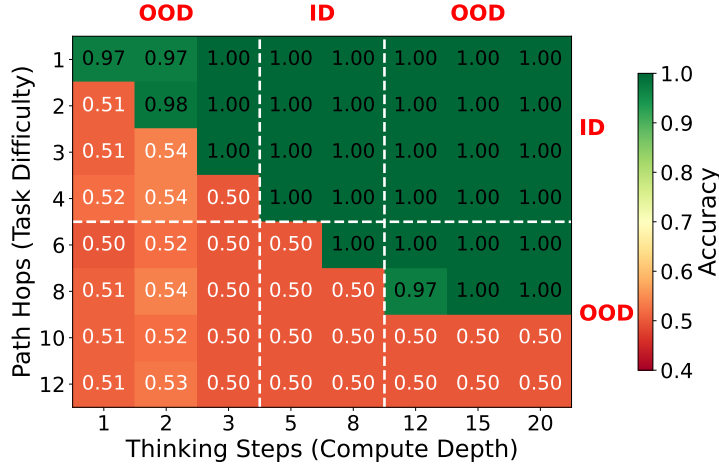


Figure 2: Accuracy heatmap for the graph reachability task. A sharp computational frontier exists along the diagonal, confirming the strict 1-hop = 1-step physical constraint enforced by adjacency masking. The model generalizes perfectly to 8 hops (OOD) with sufficient steps, but collapses abruptly to chance ($\sim 50\%$) at 10 hops. Dashed lines mark the training boundaries in both dimensions (1–5 hops; 5–8 thinking steps); cells outside these ranges are also OOD, with accuracy preserved wherever the thinking-step count meets or exceeds the required hop count.

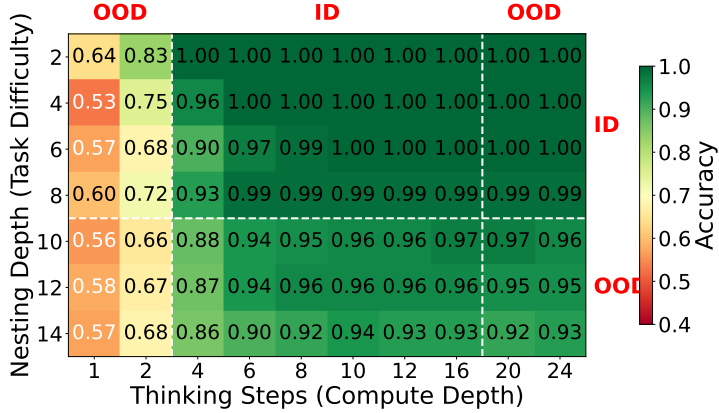


Figure 3: Accuracy heatmap for the nested boolean expression task. Compared to the graph task, the computational frontier here is more gradual. The model degrades gracefully of OOD samples, holding 90%+ accuracy up to depth 14 when given sufficient thinking steps, and remaining stable even when unrolled for 24 thinking steps (well beyond the training range of 4–16 thinking steps). Dashed lines mark the training boundaries in both dimensions (1–8 depth; 4–16 thinking steps).

4.2 Experiment II: Nested Boolean Expression Evaluation

Given a fragile nested boolean expression (e.g., $!((T \& F) | (! (T | F)))$), this task evaluates it to True or False. Training succeeds on nesting depths 1–8.

As depicted in Figure 3, we observe a gradual computational frontier. The model degrades gracefully beyond the training distribution, holding 90%+ accuracy at depth 14 ($1.75\times$ OOD). Unlike the graph task, accuracy increases monotonically with more thinking steps without collapsing, and the model does not “overthink” or degrade with excessive computation up to 24 steps (OOD in the step dimension), consistent with the identity-biased gate favoring state retention (Section 3.1).

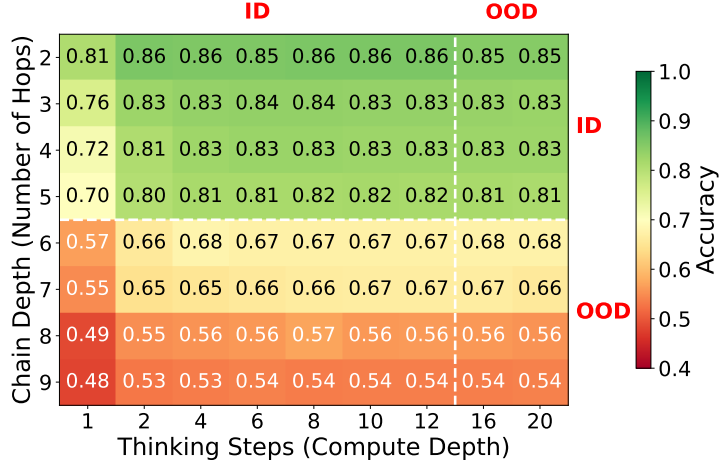


Figure 4: Accuracy heatmap for the relational composition task over unstructured text. Without task-aligned structural inductive biases (no adjacency masking; while RoPE is retained for local word order, the fully shuffled input facts ensure that ID relative positions carry no meaningful structural signal), the task exhibits a strictly monotonic increase in difficulty. The invariant reasoning core discovers latent pointer-chasing routes, staying above chance at depths 6 and 7 when additional thinking steps are provided, though accuracy falls toward chance by depth 9. Dashed lines mark the training boundaries in both the depth and step dimensions.

Why Is Degradation Graceful? Unlike the hard adjacency mask used for graph reachability, RoPE only *biases* attention toward nearby positions rather than blocking distant ones: rotating q and k by relative angles makes their dot product decay, on average, as the relative distance grows (Su et al., 2024), so even in a single step f_θ can already attend weakly to operands several nesting levels away. A model given fewer than 4 thinking steps is therefore not blind to the outer operations, unlike the graph model; it can partially propagate operand values across levels in one step, at the cost of interference between them. This is why accuracy degrades gradually instead of collapsing sharply below the required depth (Figure 3).

4.3 Experiment III: Relational Composition over Text

To test the reasoning core’s capability in a pure language modeling context, we evaluate it on a family-vocabulary relational composition task inspired by CLUTRR (Sinha et al., 2019), where the correct relation is defined by a signed-offset algebra over parent/child hops rather than literal family semantics. The input is a sequence of shuffled natural-language sentences that define relationships (e.g., Alice is the parent of Bob), padded with irrelevant distractor relations, followed by a query statement asserting a candidate relation between two entities (e.g., Alice is the sibling of Eve). The model determines whether the queried relation is correct.

Shuffling the fact sentences and padding them with irrelevant distractors already prevents shortcuts based on sentence position or sentence count. Naïvely generating chains of relations (e.g., strictly parents or strictly children), however, still allows models to cheat by counting relation words, since the label would then be a monotonic function of chain length. To enforce true algorithmic deduction (pointer chasing and offset cancellation), we generate chains using an *Apex Routing Strategy*: the path alternates a parent-hop phase (offset +1 each) and a child-hop phase (offset −1 each), so Alice → Carol → Eve composes to $(+1) + (-1) = 0$, the offset assigned to sibling. However, since $\text{offset} = 2u - K$ for a chain of length K with u up-hops, the offset and K always share the same parity, leaving a residual *parity-detection* shortcut: a model could accept or reject a candidate purely from whether its parity matches the chain length, without recovering the exact value. We close this loophole with *hard negatives* whose parity matches the correct relation (e.g., grandparent for sibling, both even), forcing genuine latent routing rather than surface pattern matching. Fixing this single-reversal shape across all instances also means that increasing the chain depth changes only the length of the signed-offset sum, not the topology the model must recognize, isolating this ability from any confound with path-shape complexity.

The model is trained on depths 2–5 with 1–12 thinking steps, and evaluated up to depth 9 and 20 thinking steps.

Table 3: In-distribution (ID) and sufficient thinking step out-of-distribution (OOD) accuracy (%) per task; **P** is parameters in millions. Recurrent models are unrolled to $T_{\max}$ at inference. Every ID and OOD number is a 3-seed mean; in the OOD column, $\pm$ is one standard deviation, omitted when it rounds to zero. The last row ablates our stability recipe (no LayerScale, no gate). The fixed-depth GNN can only be applied to the graph task; the sequence tasks have no graph to message-pass over. The Universal Transformer baseline combines weight tying with ACT adaptive halting (Dehghani et al., 2019; Graves, 2016).

Model	Graph			Nested Expr.			Family		
	P	ID	OOD	P	ID	OOD	P	ID	OOD
<i>Fixed depth</i>									
GNN	0.5	100	51	—	—	—	—	—	—
Transformer	0.8	100	57 $\pm$ 1	6.4	100	97	4.0	81	58 $\pm$ 2
<i>Weight-sharing / recurrent</i>									
Universal Transformer (ACT)	0.2	100	62	0.9	97	91 $\pm$ 2	0.9	82	57 $\pm$ 2
Depth-Recurrent (ours)	0.2	100	75 $\pm$ 0.2	1.0	100	95 $\pm$ 1	1.0	83	60 $\pm$ 1
w/o LayerScale, w/o gate (ablation)	0.2	100	71 $\pm$ 6	0.9	100	97 $\pm$ 1	0.9	82	62 $\pm$ 2

As shown in Figure 4, the results reveal three crucial insights. First, monotonic difficulty: the accuracy decreases overall as the chain depth increases. Because all parity shortcuts are removed, the model must perform genuine latent routing, which naturally becomes harder for longer chains. Second, the computational frontier: for any given depth, increasing the number of thinking steps improves accuracy up to a plateau. Third, partial OOD generalization: despite lacking task-aligned structural hints (no graph mask, and 1D relative positions provide no shortcuts for the fully shuffled sentences), the invariant core stays above chance at depths 6 and 7 (66%+ once given sufficient steps), and unrolling past the trained range (16–20 steps) neither helps nor hurts; accuracy holds flat. It, nonetheless, approaches chance by depth 9.

Why Does Accuracy Decrease Monotonically? A Worked Example Unlike graph reachability (hard adjacency mask) and nested boolean logic (RoPE’s soft bias still lets one step partially propagate across levels), the family task offers no such safety net: each additional hop is another opportunity to route to the wrong entity, with no guarantee a misrouted step is later corrected. If each hop is resolved correctly with probability $p < 1$ and errors are not selectively repaired, the probability of resolving a depth-5 chain falls off roughly as p^5 —a compounding cost rather than a threshold effect. This is why Figure 4 decreases smoothly, with neither a sharp cutoff (Figure 2) nor a high-accuracy floor (Figure 3). Additional steps still raise p substantially per hop—accuracy at depth 5 improves from 69.8% at 1 step to 81.7% at 12 steps—but even at 20 steps it plateaus below the other two tasks, since the model must still route through unstructured, shuffled text with no positional shortcut to lean on.

4.4 Comparison to Fixed-Depth and Weight-Sharing Baselines

Table 3 places our instantiation against fixed-depth and weight-sharing alternatives. Four things stand out. First, extrapolation on structured problems requires test-time depth: on graph reachability, the fixed-depth GNN sits at chance beyond the training range (1–5 hops), and the fixed-depth Transformer extrapolates just one hop further (77% at 6 hops) before falling to chance at 8 hops and beyond, whereas the recurrent variants push well past it. Second, ablating our stability recipe (last row: no LayerScale, no gate) leaves ID and mean OOD accuracy essentially unchanged but collapses training stability: the ablated model’s graph OOD swings from 62 to 76 across seeds where ours varies by 0.2. Third, on the sequence tasks, the recurrent models cluster together and match a fixed-depth Transformer that uses 4–6.4 $\times$ their parameters. Fourth, the genuine Universal Transformer—weight tying plus ACT adaptive halting and a ponder cost—is the weakest recurrent variant: on graph, it returns to chance beyond the training range (OOD 62, and exactly 50 on hops 8–12), because its learned halting saturates at ≈ 4 ponder steps where 8–12 are structurally required and the ponder penalty rewards early stopping. This undershoot is milder on the sequence tasks (91 on logic, 57 on family). An externally specified step count T avoids this coupling between halting and optimization (Section 5.5).

4.5 Silent Thinking vs. Intermediate Supervision

A natural hypothesis for improving algorithmic reasoning is to apply *intermediate supervision* (Lee et al., 2015; Newell et al., 2016; Wu et al., 2022): computing and averaging the loss at every thinking step. We compare our silent-thinking objective against an intermediate-supervision baseline on the graph reachability task, which is ideal for diagnosis

Table 4: Supervision strategies on graph reachability. Step-1 Acc. 12-hop measures accuracy after a single thinking step on 12-hop queries, where the adjacency mask bounds true accuracy at $\approx 50\%$; any excess indicates a statistical shortcut. Suff.-step OOD averages the best accuracy over the OOD hop counts $\{6, 8, 10, 12\}$ once given enough thinking steps, including 12-hop, which stays at chance for every configuration; 10-hop is the deepest path any configuration solves.

Objective	Step-1 Acc. 12-hop $\downarrow$	10-hop OOD $\uparrow$	Suff.-step OOD $\uparrow$	Train acc. (step-mean)
Silent thinking ($\alpha = 0$)	50.0%	84%	83.6%	87.6%
Light per-step ($\alpha = 0.1$)	69.4%	50%	75.0%	94.1%
Full per-step ($\alpha = 1$)	65.4%	51%	74.8%	96.9%

because the adjacency mask provides a physically verifiable ground truth: a model taking k steps can only aggregate information from k hops away.

We interpolate between two objectives using parameter α .

$$\mathcal{L}(\alpha) = (1 - \alpha) \text{CE}(\hat{y}^{(T)}, y) + \frac{\alpha}{T} \sum_{t=1}^T \text{CE}(\hat{y}^{(t)}, y), \quad (12)$$

where $\hat{y}^{(t)}$ is the readout applied to $H^{(t)}$, so $\alpha = 0$ recovers silent thinking; $\alpha = 1$ is per-step supervision. We deliberately widen the dimension to $d = 256$ because with smaller capacity ($d = 128$), *neither* objective propagates past 8 hops.

Table 4 is the average over 5 runs. The intermediate supervision exhibits an apparent anomaly: it achieves roughly 65–70% accuracy on 12-hop paths after only a *single* thinking step. Under strict topological masking, a 1-step model has no information about nodes 12 hops away. This validates that the model abandons genuine message-passing and likely learns statistical heuristics, such as estimating reachability from graph density or the source node’s degree, for early-step optimization.

Varying α produces a threshold effect, not a graded trade-off. Silent thinking is the only setting that turns the extra width into deeper propagation: it solves 10-hop paths (84% at $d = 256$; 88–100% for all 5 seeds at $d = 384$), and its sufficient-step OOD accuracy grows with capacity (75% $\rightarrow$ 83.6% $\rightarrow$ 87% at $d = 128/256/384$). This property disappears even when $\alpha = 0.1$: 10-hop accuracy falls to chance for every seed. Full per-step supervision ($\alpha = 1$) shows the same ceiling: OOD accuracy stays flat across the same widths (75.1% $\rightarrow$ 74.8% $\rightarrow$ 75.1%) even as its mean per-step training accuracy climbs monotonically (94.7% $\rightarrow$ 96.9% $\rightarrow$ 98.3%).

We attribute this to a *bandwidth occupation* failure mode. When penalized at Step 1, the model faces a choice: adopt an “honest strategy” (accepting 50% early accuracy to learn deep propagation) or a “shortcut strategy” (learning shallow heuristics for immediate reward). Intermediate supervision makes the shortcut irresistible. Once committed to these heuristics, the model loses the incentive to develop the true sequential algorithm and fails on deep OOD paths even when granted many steps at test time: as shown above, added width lets it fit the intermediate readouts more tightly without translating into deeper propagation.

5 Analysis and Discussion

5.1 Precise vs. Robust Generalization

The most revealing finding is the difference in generalization behavior across the three tasks under the same reasoning core. Graph reachability (adjacency masking) is precise but brittle: the strong structural prior requires one thinking step per hop, so OOD generalization is perfect up to $1.6\times$ and then collapses, though added width extends this reach (Section 4.5). It is also the one task where a fixed-depth Transformer, even with several times more parameters, extrapolates only one hop before falling to chance (Table 3). Boolean logic (RoPE) is approximate but robust: the relative-positioning bias allows flexible propagation patterns, giving graceful degradation ($> 90\%$ accuracy at $1.75\times$ OOD). Unstructured text is the hardest to optimize but degrades slowly rather than collapsing, reaching modest but above-chance OOD accuracy by chasing pointers purely through latent representations.

This points to a fundamental trade-off: stronger inductive biases yield precise in-distribution behavior but brittle extrapolation, while weaker biases sacrifice precision for robustness.

5.2 Depth-Embedding Extrapolation Is Not Confounded

Because the step count T is sampled from a bounded range during training, thinking steps beyond that range use depth embeddings e_t that never receive a gradient update and remain at their random initialization, a potential confound for the OOD claims in Sections 4.1 to 4.3: apparent stability there could reflect these embeddings being effectively ignored rather than genuine architectural generalization. We tested this by substituting the untrained embeddings at inference with (i) zeros, (ii) the last in-distribution embedding (called clamping), and (iii) three independent random reinitializations, then re-evaluating each trained checkpoint on the same beyond-range grid. We found that the four variants (baseline/zero/clamp/reinit) agree closely on all three tasks in this region—accuracy differs by at most 0.5 percentage points across variants (e.g., 0.744 vs. 0.739 for graph) and by ≤ 0.2 points across reinitialization seeds. The gate value $z^{(t)}$ also shows no discontinuity at the train/beyond-range boundary on any task. We conclude that the reported beyond-range stability is a genuine property of the identity-biased recurrence rather than an artifact of the specific untrained embedding values. As a byproduct, this also confirms that the graph task’s collapse beyond 8 hops (Section 4.1, at $d = 128$) is not rescued by clamping to a trained embedding.

5.3 Why Negative Gate Bias Prevents Vanishing Gradients

The intuitive justification for $b_z = -2.0$ (Section 3.1) is that it makes the model retain 88% of its previous state by default; here we make precise why this stabilizes gradient flow across 20+ unrolled steps. Differentiating the gated update $H^{(t)} = z^{(t)} \odot \tilde{H}^{(t)} + (1 - z^{(t)}) \odot H^{(t-1)}$ and, following standard gated-recurrence gradient analyses (Gers et al., 2000), treating $z^{(t)}$ as approximately constant with respect to $H^{(t-1)}$ over a single step, the Jacobian decomposes as

$$\frac{\partial H^{(t)}}{\partial H^{(t-1)}} \approx \text{diag}(1 - z^{(t)}) + \text{diag}(z^{(t)}) \frac{\partial \tilde{H}^{(t)}}{\partial H^{(t-1)}}. \quad (13)$$

The first term bypasses f_θ ; the second routes through the reasoning block and is damped by LayerScale, whose $\gamma_{\text{attn}}, \gamma_{\text{ffn}}$ are initialized near zero (10^{-4} , Section 3.1) so that this branch contributes negligible signal early in training. Chaining the identity-path term across all T steps, the gradient of the silent-thinking loss with respect to the initial state becomes

$$\frac{\partial \mathcal{L}}{\partial H^{(0)}} = \frac{\partial \mathcal{L}}{\partial H^{(T)}} \prod_{t=1}^T \text{diag}(1 - z^{(t)}) + R, \quad (14)$$

where R collects the remaining chain-rule terms that route through f_θ at least once at some step. At initialization $W_z \approx 0$, so $z^{(t)} \approx \sigma(b_z)$ is approximately constant across steps and positions. With $b_z = -2.0$, $(1 - z^{(t)}) \approx 0.88$, so this path attenuates the gradient by only 0.88^T (e.g., $0.88^{20} \approx 0.078$), a modest decay that keeps the gradient in a numerically healthy range, in contrast to the symmetric initialization $b_z = 0$, where $(1 - z^{(t)}) \approx 0.5$ and the same calculation gives $0.5^{20} \approx 9.5 \times 10^{-7}$, nearly five orders of magnitude smaller in the vanishing-gradient regime. Because this identity path does not route through the LayerScale-suppressed branch, it remains the dominant channel carrying gradient signal back to $H^{(0)}$ and the perception interface. LSTMs justify large forget-gate bias initialization similarly (Gers et al., 2000): a near-linear self-connection lets the gradient signal pass through many steps largely unchanged. We adapt that argument to a Transformer-based recurrent core. As training progresses and W_z is learned, $z^{(t)}$ departs from this constant profile to encode instance-specific update decisions.

5.4 Computational and Memory Complexity

The following comparison focuses on the cost of *thinking*, not producing the final answer, which is a single token in both paradigms. Throughout, T denotes the depth of this reasoning process: T recurrence steps for our vertical CoT, or T reasoning tokens generated before the answer for horizontal CoT.

Because f_θ is a single block applied recurrently, its parameter count, $\Theta(d^2 + d d_{\text{ff}})$, and per-step compute, $\Theta(L^2 d + L d^2 + L d d_{\text{ff}})$ (identical to one Transformer layer), are both independent of T ; unrolling for T steps costs $\Theta(T(L^2 d + L d^2 + L d d_{\text{ff}}))$ total FLOPs, linear in T , with L unchanged.

Horizontal CoT, in contrast, must generate these T thinking tokens one at a time, each requiring a full pass through all N layers. With a KV-cache, attending to the growing context costs $\Theta(N(TLd + T^2 d))$ total attention FLOPs across the T tokens, quadratic in T and scaled by the backbone depth N , unlike our linear-in- T , N -independent recurrence. On memory, horizontal CoT’s KV-cache holds $L + T$ tokens across all N layers, $\Theta((L + T)Nd)$, while our recurrence discards each step’s keys and values once $H^{(t)}$ is updated and so holds L tokens for a single block, $\Theta(Ld)$, independent of T . Their ratio, $(1 + T/L)N$, separates the two sources of the saving: the N is weight sharing and stays constant,

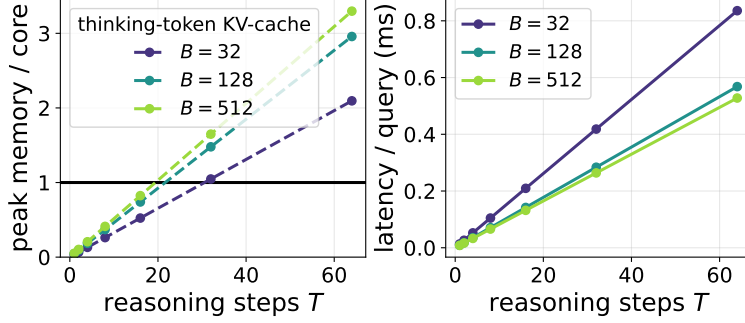


Figure 5: Measured inference cost of the reasoning core as the step count T grows. Left: the key–value cache T thinking tokens add to a depth-24, $d=1024$ backbone (GPT-2 medium, fp16; prompt excluded), as a multiple of the core’s T -independent footprint (the solid line at 1); linear in T , reaching $2.1\text{--}3.3\times$ by $T=64$. Right: latency per query, linear in T and falling as the batch grows.

while the $1 + T/L$ is what emitting no tokens buys, which is independent of width, depth, and batch size, unbounded as reasoning lengthens, and correspondingly small when reasoning is short relative to the prompt.

The two paradigms also differ in *latency*: since the N layers within one token’s forward pass are sequentially dependent, and each of the T tokens must wait for the previous one to be produced, horizontal CoT’s critical path spans $\Theta(TN)$ sequential layer evaluations. Our recurrence applies a single block T times, so its critical path is $\Theta(T)$.

Figure 5 confirms this scaling across serving batch sizes. The left panel compares memory. The reasoning core’s peak memory does not change with T at inference, so we take it as the unit (the solid line at 1). The dashed curves show the key–value cache the backbone builds up while generating T thinking tokens, measured on GPT-2 medium (Radford et al., 2019) ($N=24$). We count only the thinking tokens: the prompt occupies memory in both models, so it cancels out. This cache grows linearly in T , passing the core’s entire footprint at $T\approx 19\text{--}31$ and reaching $2.1\text{--}3.3\times$ it by $T=64$. The right panel reports latency per query. It is linear in T , as the $\Theta(T)$ critical path predicts, and falls as the batch grows: 0.84 ms at $B=32$ against 0.53 ms at $B=512$, both at $T=64$, since a wider batch amortizes each block application over more queries.

5.5 Limitations and Future Work

Scale We deliberately study reasoning cores of fewer than 1M parameters, isolating the recurrence mechanisms from large-scale pretraining confounds and keeping the depth at which each task fails measurable. The cost model of Section 5.4 is independent of core width, so it carries to deployment scale. Whether identity-biased gating and LayerScale still suffice as d , d_{ff} , and L grow by orders of magnitude remains open.

Manually Designed Perception Interfaces Our three interfaces (topological masking, RoPE, standard attention) were hand-selected per task; a more general system should unify these choices into one interface, e.g., a learning block that interpolates between strict priors and unconstrained attention.

Fixed, Externally Specified Depth At inference time, we treat the recurrence step count T as an external hyperparameter, which lets us separate depth generalization from stopping-time learning. A practical system would benefit from a lightweight halting module—in the spirit of ACT (Graves, 2016) but applied at the sequence level rather than per token—that allocates more steps to harder instances without a preset budget. This is harder than it looks. When we train an ACT Universal Transformer, the learned halting budget is calibrated to the training distribution and does not extrapolate: on graph reachability it saturates near 4 ponder steps, so OOD instances needing 8 – 12 fall back to chance (Table 5). The undershoot is milder where a constant step budget suffices (nested logic) and reappears where required depth grows with difficulty (family).

Integration with Pretrained LLMs Moving the reasoning core from task-specific models to a pretrained LLM raises new design considerations:

- **Placement:** designate a single existing layer or small contiguous block as the recurrent core and loop it in place, rather than sharing weights across *all* layers.

Table 5: Ponder steps in the OOD regime vs. the depth the task requires. When they mismatch, OOD accuracy degrades toward chance.

Task	Learned ponder	Steps required	OOD acc.
Graph (hops 8–12)	≈ 4.4	8–12	50
Nested logic (d. 10–14)	≈ 10.2	≈ 10	91
Family (hops 6–9)	≈ 6.3	grows with depth	57

- Warm-starting: initialize the gate to reproduce the pretrained layer’s single-pass behavior at $T = 1$, introducing extra steps gradually during fine-tuning.
- Parameter-efficient adaptation: leave the pretrained weights frozen and carry the recurrence entirely in a LoRA (Hu et al., 2022) side branch, or a higher-capacity variant such as CeRA (Chen, 2026), training only the adapters with the new depth embeddings, gate, and LayerScale parameters.
- Trigger mechanism: decide when to invoke the recurrent block instead of unrolling it everywhere, connecting to the halting-policy question above (Wei et al., 2022; Goyal et al., 2024; Hao et al., 2025).

6 Conclusion

We study *vertical Chain-of-Thought*: adding test-time reasoning compute by iterating a shared-weight block in latent space rather than generating tokens, so that depth costs flat memory and linear latency. Using a stable instantiation—silent thinking, LayerScale, and identity-biased recurrence unrolled over 20+ steps—we characterize this mechanism on graph reachability, nested boolean logic, and unstructured relational composition. Test-time depth recurrence extrapolates beyond the training range: it succeeds on the graph reachability task where fixed-depth Transformers fail, and it comes within two points of fixed-depth models several times larger on the two sequence tasks. The contrast between the three tasks reveals how perception interfaces shape the generalization profile of a shared reasoning core, and we find that per-step supervision, a common recipe for deep iterative models, undermines this extrapolation. By “thinking deeper, not longer,” models can spend variable computational depth without consuming the context window or the key–value cache, allowing reasoning across large query batches.

GenAI Usage Disclosure

The authors used Gemini and Claude to improve language and readability. The authors used Claude Code to assist in coding and experimenting. The authors reviewed and edited the content and the code as needed and take full responsibility for the content.

Acknowledgments and GenAI Usage Disclosure

We acknowledge support from the National Science and Technology Council of Taiwan under grant number 113-2221-E-008-100-MY3.

References

- Bai, S., Kolter, J. Z., and Koltun, V. (2019). Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Chen, H.-H. (2026). Cera: Overcoming the linear ceiling of low-rank adaptation via capacity expansion. In *Conference on Language Modeling (CoLM)*.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. (2019). Universal transformers. In *International Conference on Learning Representations (ICLR)*.
- Dong, Y., Cordonnier, J.-B., and Loukas, A. (2021). Attention is not all you need: Pure attention loses rank doubly exponentially with depth. In *International Conference on Machine Learning (ICML)*, pages 2793–2803. PMLR.
- Feng, G., Zhang, B., Gu, Y., Ye, H., He, D., and Wang, L. (2023). Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- Gers, F. A., Schmidhuber, J., and Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471.
- Goyal, S., Ji, Z., Rawat, A. S., Menon, A. K., Kumar, S., and Nagarajan, V. (2024). Think before you speak: Training language models with pause tokens. In *International Conference on Learning Representations (ICLR)*.
- Graves, A. (2016). Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. (2025). Training large language models to reason in a continuous latent space. In *International Conference on Learning Representations (ICLR)*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022). Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 22199–22213.
- Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. (2020). ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*.
- Lee, C.-Y., Xie, S., Gallagher, P., Zhang, Z., and Tu, Z. (2015). Deeply-supervised nets. In *Artificial Intelligence and Statistics (AISTATS)*, pages 562–570.
- Merrill, W. and Sabharwal, A. (2024). The expressive power of transformers with chain of thought. In *International Conference on Learning Representations (ICLR)*.
- Newell, A., Yang, K., and Deng, J. (2016). Stacked hourglass networks for human pose estimation. In *European Conference on Computer Vision (ECCV)*, pages 483–499. Springer.
- Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318. PMLR.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). Language models are unsupervised multitask learners. Technical report, OpenAI.
- Sinha, K., Sodhani, S., Dong, J., Pineau, J., and Hamilton, W. L. (2019). CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Snell, C. V., Lee, J., Xu, K., and Kumar, A. (2025). Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568(C).
- Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., and Jégou, H. (2021). Going deeper with image transformers. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 32–42.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS)*, Red Hook, NY, USA. Curran Associates Inc.
- Wu, D. Y., Lin, D.-N., Chen, V. F., and Chen, H.-H. (2022). Associated learning: a methodology to decompose end-to-end backpropagation on cnn, rnn, and transformer. In *International Conference on Learning Representations (ICLR)*.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T.-Y. (2020). On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. JMLR.org.